

Lecture 28: Machine Learning Continued

Heather Mattie, PhD

December 16, 2025

Recipe of the Day!

Butternut Squash Risotto



Announcements

- No lab this week
- Project files due Thursday, December 18th by 12pm (noon)
- No TF office hours this week – just Heather's, today 1-2pm

Machine Learning

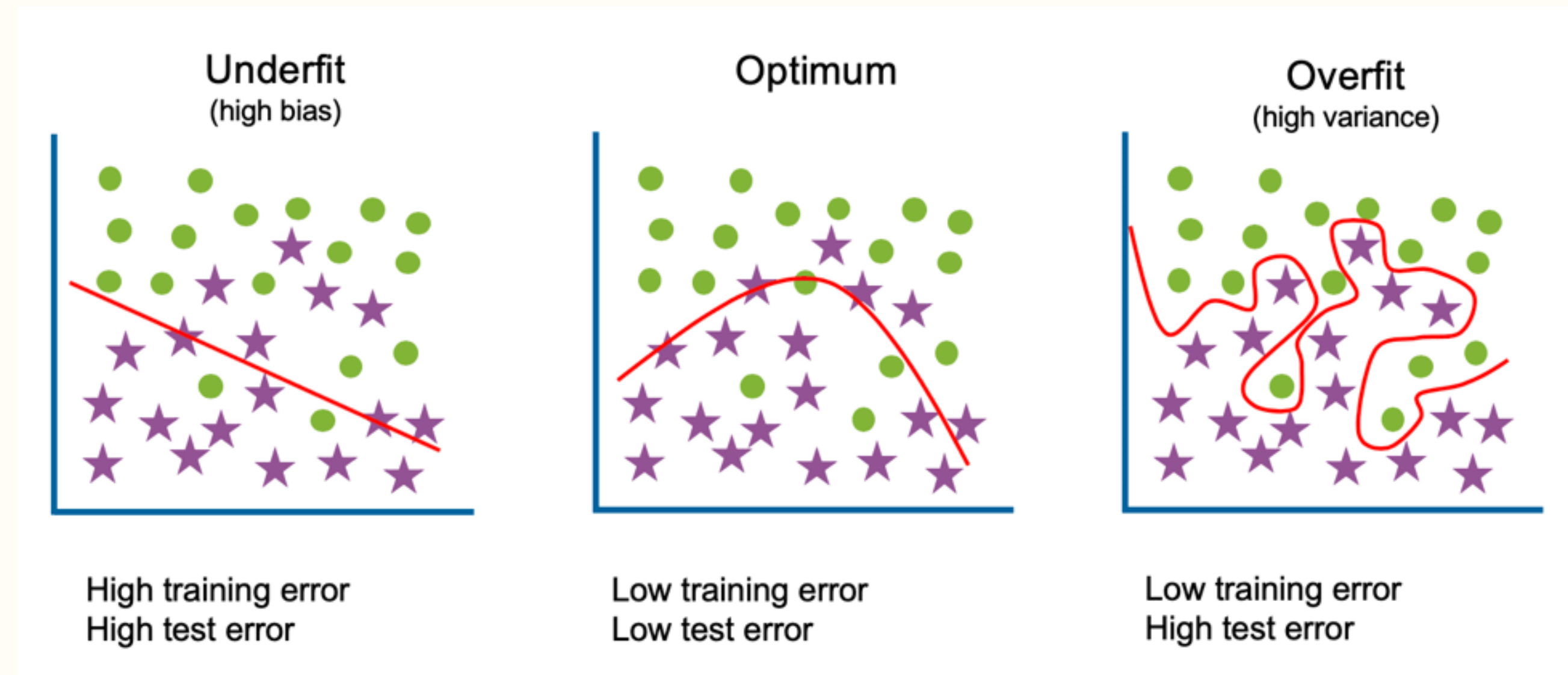
Principal Component Analysis (PCA)

Heather Mattie, PhD

Regularization

Regularization is a technique used in machine learning to prevent overfitting by decreasing variance. Two common methods:

- **Shrinkage**: LASSO and ridge regression
- **Dimension reduction**: principal component analysis (PCA)

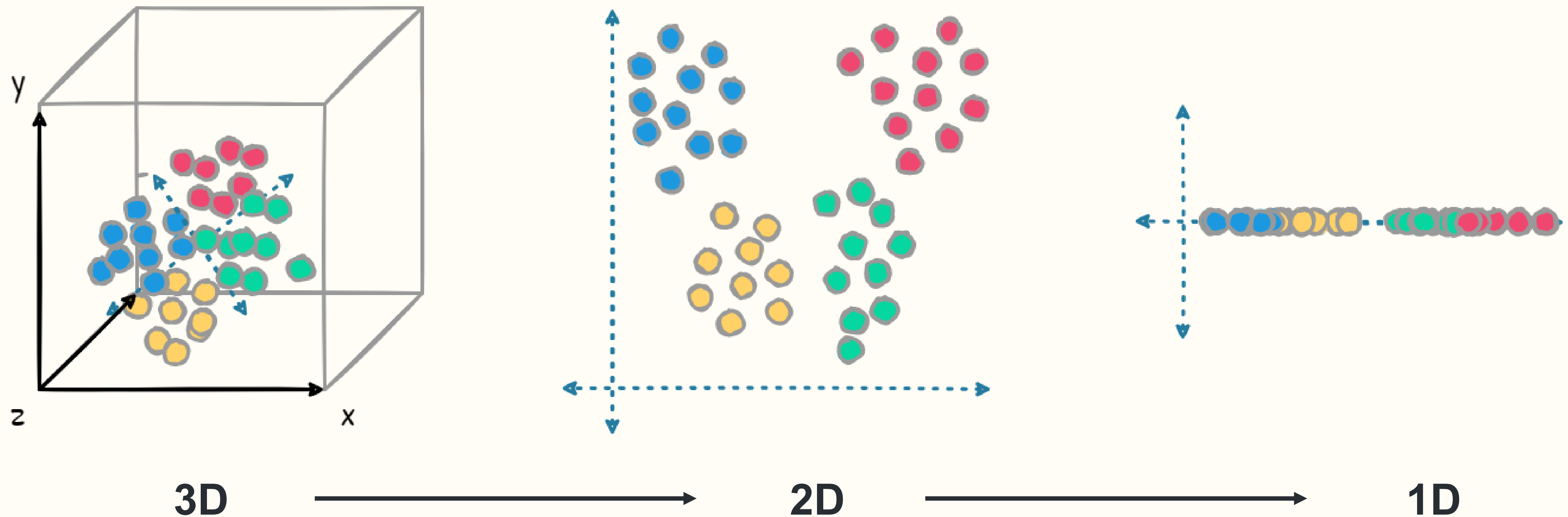


Dimensionality reduction

Dimensionality reduction is the process of decreasing the number of input features in a dataset while preserving essential information.

- Transforms high-dimensional data into a lower-dimensional space
- Done to simplify models, improve performance, reduce training time, and combat the "curse of dimensionality".

Dimensionality reduction

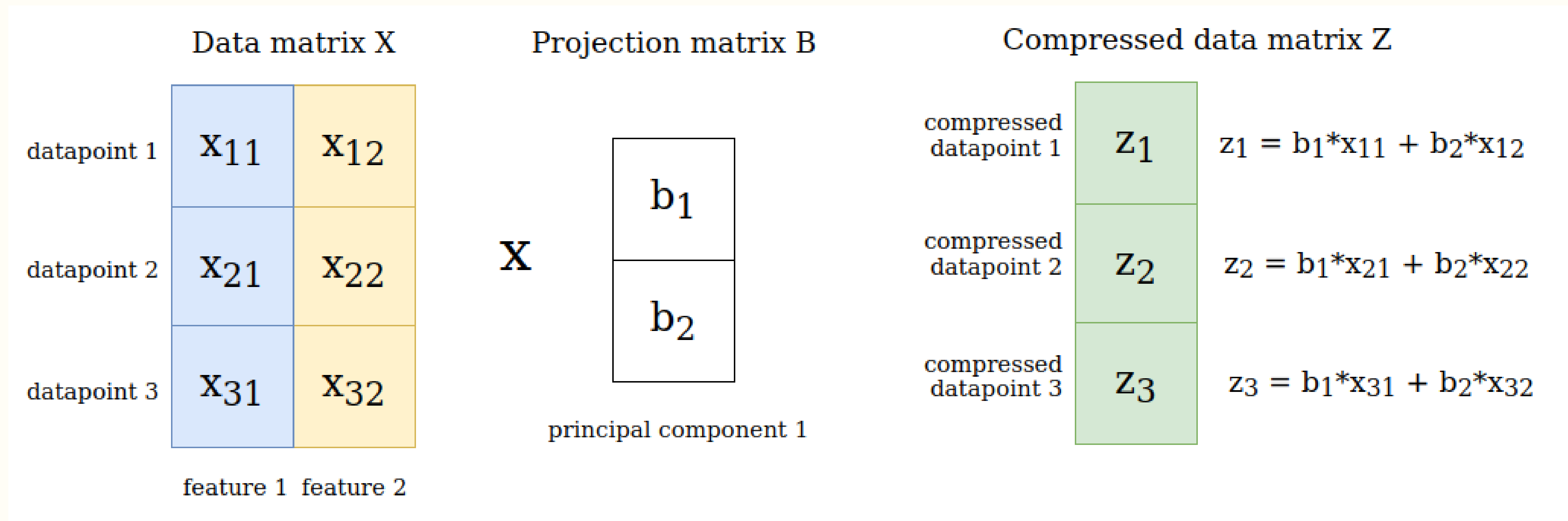


Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms variables into fewer, uncorrelated components, called principal components (PCs), to capture most data variance.

- Creates new variables (PCs) that are linear combinations of the original variables, ordered by the variance they explain
- All PCs are uncorrelated (orthogonal)
- First PC (PC_1) captures the most variance in the data
- Unsupervised method that focuses on data structure – does not involve the outcome variable

Principal Component Analysis (PCA)



Example: Breast Cancer

The Breast Cancer Coimbra dataset measures nine quantitative clinical features for 64 patients with breast cancer and 52 healthy controls.

- Age (years)
- BMI (kg/m²)
- Glucose (mg/dL)
- Insulin (μU/mL)
- HOMA (Homeostasis model assessment for insulin resistance)
- Leptin (ng/mL)
- Adiponectin (μg/mL)
- Resistin (ng/mL)
- MCP.1 (pg/dL)

Example: Breast Cancer

```
# Read in breast cancer data
bc_dat = read.csv("breast_cancer.csv")

head(bc_dat)
```

##	Age	BMI	Glucose	Insulin	HOMA	Leptin	Adiponectin	Resistin	MCP.1
## 1	48	23.50000	70	2.707	0.4674087	8.8071	9.702400	7.99585	417.114
## 2	83	20.69049	92	3.115	0.7068973	8.8438	5.429285	4.06405	468.786
## 3	82	23.12467	91	4.498	1.0096511	17.9393	22.432040	9.27715	554.697
## 4	68	21.36752	77	3.226	0.6127249	9.8827	7.169560	12.76600	928.220
## 5	86	21.11111	92	3.549	0.8053864	6.6994	4.819240	10.57635	773.920
## 6	49	22.85446	92	3.226	0.7320869	6.8317	13.679750	10.31760	530.410
##	Classification								
## 1			1						
## 2			1						
## 3			1						
## 4			1						
## 5			1						
## 6			1						



Example: Breast Cancer

- Recode the outcome (Classification) to 0: health control and 1: cancer patient.
- Split data into training and test sets. A 50/50 split is used here since the same size is so small.
- Create a list of predictor names to help index the training set features later.

```
# Predictor variables
predictors = setdiff(names(bc_dat), "Classification")

# Recode Classification as a factor with 1 for cases and 0 for controls
bc_dat$Classification = factor(ifelse(bc_dat$Classification == 2, 1, 0))

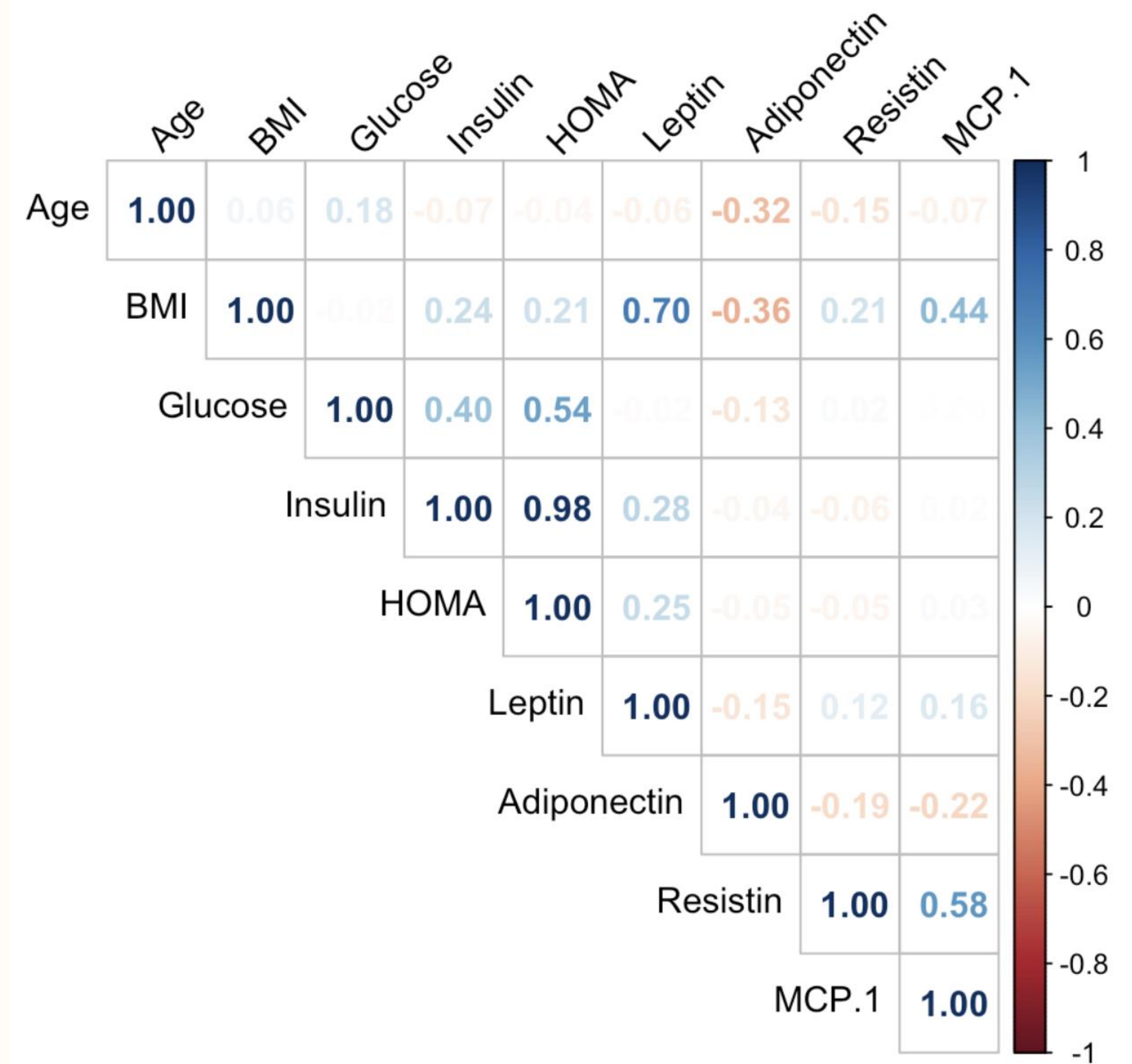
# Split the data into a training set and a test set
set.seed(219)
train_idx = createDataPartition(bc_dat$Classification,
                                times = 1, p = 0.5, list = FALSE)

bc_train = bc_dat[train_idx, ]
bc_test = bc_dat[-train_idx, ]
```

Multicollinearity

Most of the features are correlated with each other.

```
library(corrplot)
corr_matrix <- cor(bc_train[,predictors],
                  method = "spearman")
corrplot(corr_matrix,
         method = "number",
         type = "upper",
         tl.col = "black",
         tl.srt = 45)
```



prcomp

The **prcomp** function performs PCA.

- Only include **predictors** from the training set – not the outcome variable
- Standardize data using **center = TRUE** and **scale = TRUE**

```
pca = prcomp(bc_train[,predictors], center = TRUE, scale = TRUE)
```

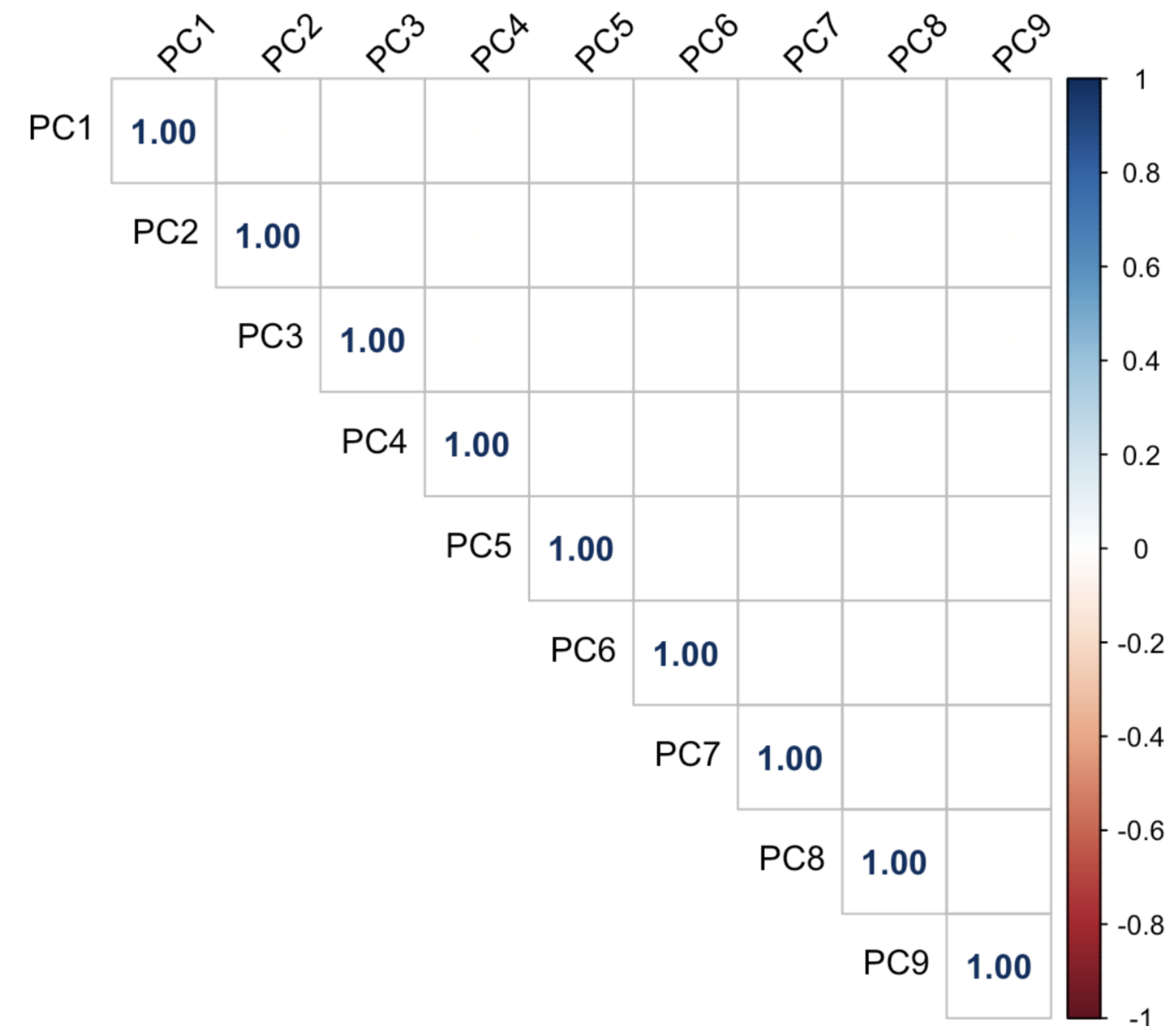

PC correlation

None of the PCs are correlated, as expected from the PCA method.

```
corr_matrix <- cor(data.frame(pca$x))  
round(corr_matrix, digits = 2)
```

##		PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
##	PC1	1	0	0	0	0	0	0	0	0
##	PC2	0	1	0	0	0	0	0	0	0
##	PC3	0	0	1	0	0	0	0	0	0
##	PC4	0	0	0	1	0	0	0	0	0
##	PC5	0	0	0	0	1	0	0	0	0
##	PC6	0	0	0	0	0	1	0	0	0
##	PC7	0	0	0	0	0	0	1	0	0
##	PC8	0	0	0	0	0	0	0	1	0
##	PC9	0	0	0	0	0	0	0	0	1

```
corrplot(corr_matrix, method = "number", type = "upper", tl.col = "black", tl.srt = 45)
```



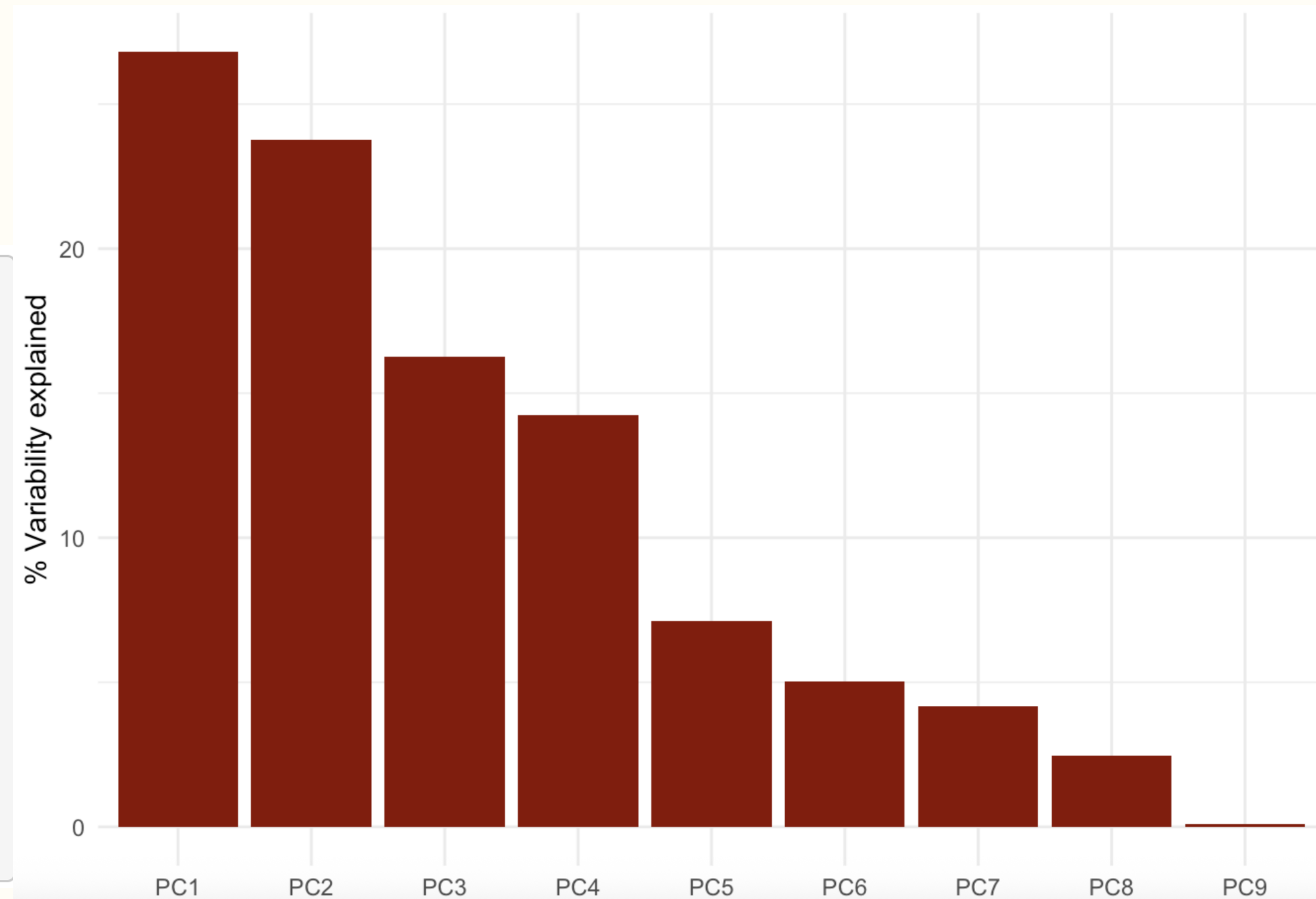
Variance explained

The % variance explained by each PC can be visualized using a bar chart.

```
# % variance explained (eigenvalues / total)
var_explained <- (pca$sdev^2) / sum(pca$sdev^2) * 100

scree_df <- tibble(
  PC = factor(paste0("PC", seq_along(var_explained)),
              levels = paste0("PC", seq_along(var_explained))),
  pct = var_explained
)

ggplot(scree_df, aes(x = PC, y = pct)) +
  geom_col(fill = "darkred") +
  labs(x = "Principal component",
       y = "% Variability explained") +
  theme_minimal()
```



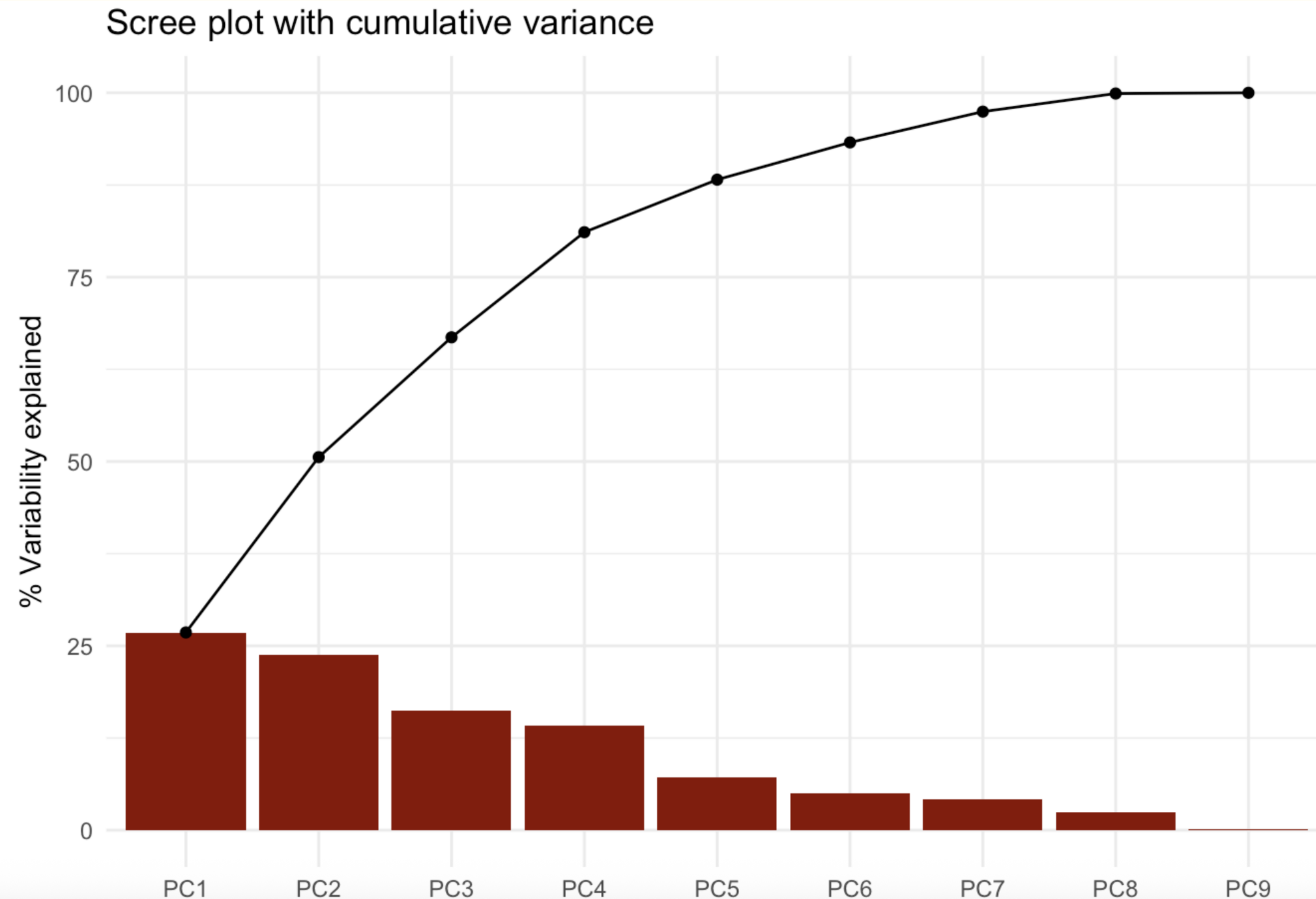
Variance explained

The cumulative variance explained (line graph) is useful for choosing how many PCs to use for model training.

```
var_explained <- (pca$sdev^2) / sum(pca$sdev^2) * 100

scree_df <- tibble(
  PC_num = seq_along(var_explained),
  PC = factor(paste0("PC", PC_num), levels = paste0("PC", PC_num)),
  pct = var_explained,
  cum_pct = cumsum(var_explained)
)

ggplot(scree_df, aes(x = PC)) +
  geom_col(aes(y = pct), fill = "darkred") +
  geom_line(aes(y = cum_pct, group = 1)) +
  geom_point(aes(y = cum_pct)) +
  labs(x = "Principal component",
       y = "% Variability explained",
       title = "Scree plot with cumulative variance") +
  theme_minimal()
```



Principal Component Regression (PCR)

Principal Component Regression (PCR) uses the PCs from PCA as new predictors in a regression model.

- Addresses multicollinearity by creating a stable, smaller set of predictors
- PCR is supervised, using PCs to predict a known outcome
- Interpretability is lost

Benchmark model

A logistic regression model with all of the original predictors as features in the model will be the baseline/benchmark model to compare the PCR models to.

```
fit_original = glm(Classification ~ .,  
                   data = bc_train,  
                   family = "binomial")  
  
fit_original_preds = predict(fit_original,  
                             newdata = bc_test,  
                             type = "response")
```


Benchmark model performance

The model is better at predicting health controls (higher specificity)

- The AUC is 0.79

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0 23 13
##           1  3 19
##
##           Accuracy : 0.7241
##           95% CI : (0.591, 0.8334)
##           No Information Rate : 0.5517
##           P-Value [Acc > NIR] : 0.005315
##
##           Kappa : 0.4617
##
##           Mcnemar's Test P-Value : 0.024449
##
##           Sensitivity : 0.5938
##           Specificity : 0.8846
```

```
# AUC
res_original$auc
```

```
## [1] 0.7932692
```

Logistic regression with PC1

PC₁ explains the most variability in the data.

- Fit a logistic regression model with PC₁ as the only predictor

```
# Add training set outcome to PCs
bc_train_pca = data.frame(pca$x, Classification = bc_train$Classification)

# Train PC1 model
fit_PCA1 = glm(Classification ~ PC1,
               data = bc_train_pca,
               family = "binomial")
```

Transforming the test set

Before prediction, the test set must be transformed in the same way the training data was during PCA.

- The **predict** function with the PCA function as the model does this automatically

```
# Transform the test set using the PCA from the training set
pca_pred = predict(pca, newdata = bc_test)

# Add test set outcome to transformed predictors
bc_test_pca = data.frame(pca_pred,
                        Classification = bc_test$Classification)
```


Making predictions

After transforming the test set predictors, prediction can be performed in the usual way.

```
fit_PCA1_preds = predict(fit_PCA1,  
                        newdata = bc_test_pca,  
                        type = "response")
```


PC₁ model performance

The model is better at predicting who is a cancer patient (higher sensitivity)

- AUC is lower at 0.70

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 12   8
##           1 14  24
##
##               Accuracy : 0.6207
##               95% CI : (0.4837, 0.7449)
##       No Information Rate : 0.5517
##       P-Value [Acc > NIR] : 0.1779
##
##               Kappa : 0.2162
##
##   Mcnemar's Test P-Value : 0.2864
##
##               Sensitivity : 0.7500
##               Specificity : 0.4615
```

```
res_pca1$auc
```

```
## [1] 0.7043269
```

Logistic regression with PC_1 - PC_5

$PC_1 - PC_5$ explain almost 90% of the variability in the data.

- Fit a logistic regression model with PC_1 - PC_5 as the predictors

```
# Train PC1-PC5 model
fit_PCA5 = glm(Classification ~ PC1 + PC2 + PC3 + PC4 + PC5,
               data = bc_train_pca,
               family = "binomial")

fit_PCA5_preds = predict(fit_PCA5,
                        newdata = bc_test_pca,
                        type = "response")
```

PC₁-PC₅ model performance

The model is more balanced in terms of sensitivity and specificity.

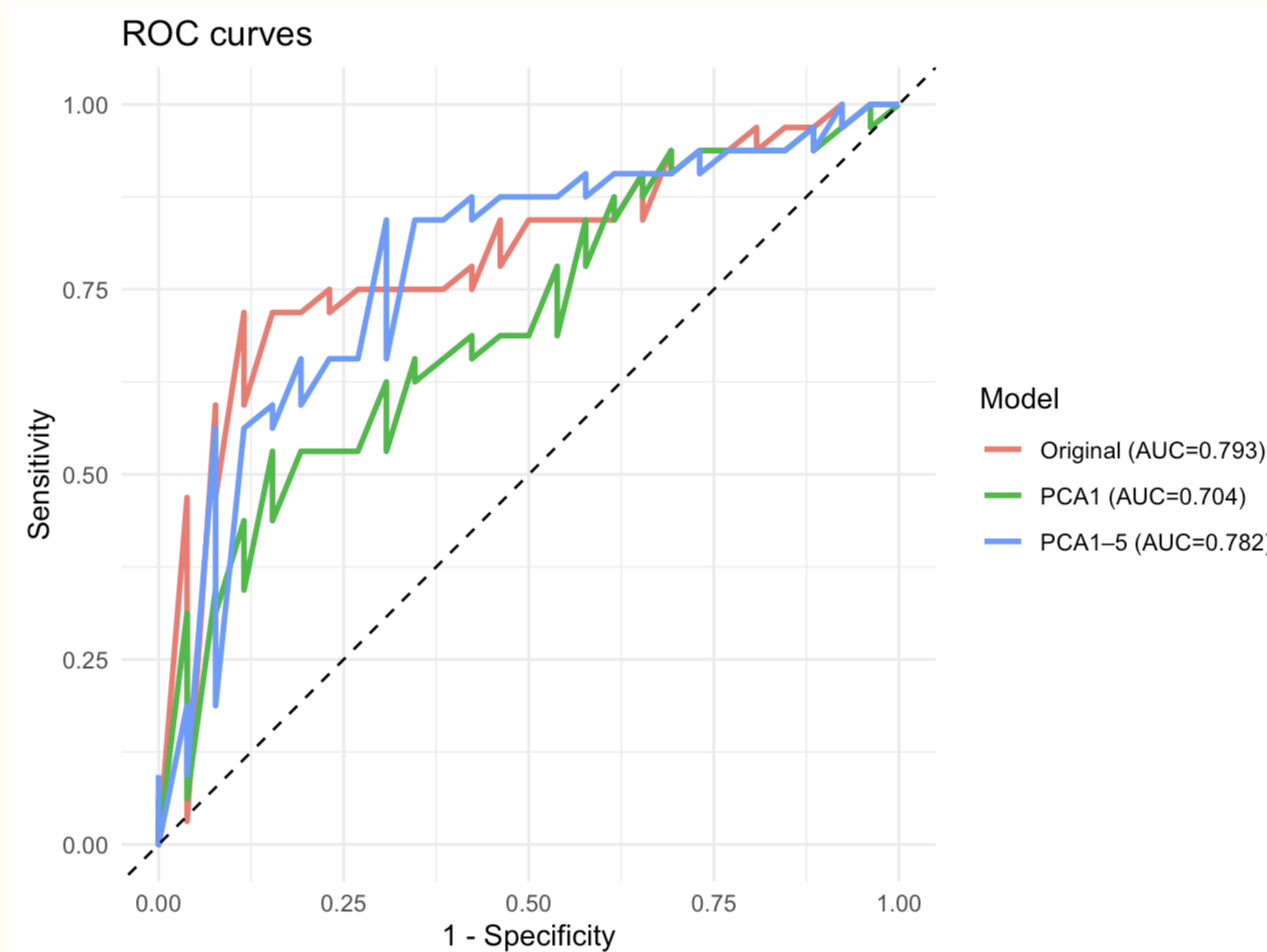
- AUC = 0.78

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0 18   8
##           1   8  24
##
##           Accuracy : 0.7241
##           95% CI : (0.591, 0.8334)
##           No Information Rate : 0.5517
##           P-Value [Acc > NIR] : 0.005315
##
##           Kappa : 0.4423
##
##           Mcnemar's Test P-Value : 1.000000
##
##           Sensitivity : 0.7500
##           Specificity : 0.6923
```

res_pca5\$auc

```
## [1] 0.7824519
```


Model comparison



Model comparison

Model	Overall accuracy	Sensitivity	Specificity	Balanced accuracy	AUC
Original	0.724	0.594	0.885	0.739	0.793
PC ₁	0.621	0.750	0.462	0.606	0.704
PC ₁ - PC ₅	0.724	0.750	0.692	0.721	0.782

The model trained with the first 5 PCs performs on par with the original model using all 9 predictors.

- May want to use the PC₁ - PC₅ model since it has fewer predictors
- May not want to use the PC₁ - PC₅ model since interpretability is lost

PCA Summary

PCA is an unsupervised dimensionality-reduction method

- Transforms correlated variables into uncorrelated principal components
- Each component is a linear combination of the original predictors
- Components are ordered by the amount of variance they explain

Often used to:

- Reduce dimensionality
- Address multicollinearity
- Visualize high-dimensional data

PCR Summary

PCR is a regression approach that uses principal components as predictors.

Steps:

1. Apply PCA to the predictor variables
2. Select a subset of principal components
3. Fit a regression model using those components

Number of components typically chosen using:

1. Scree plots (elbow plots)
2. Percent variance explained
3. Cross-validation

PCR Summary

PCR Strengths

- Reduces overfitting
- Improves stability (i.e., reduces variance) when predictors are highly correlated

PCR Limitations

- Principal components are linear combinations of many variables
- Regression coefficients no longer correspond to original predictors
- Harder to translate results into clinical or public health insights

Machine Learning

K-means clustering

Heather Mattie, PhD

Unsupervised learning

Supervised ML methods:

- The outcome y is known
- The goal is to learn a relationship between predictors (x) and y , and to **make predictions** of y for new observations

Unsupervised ML methods:

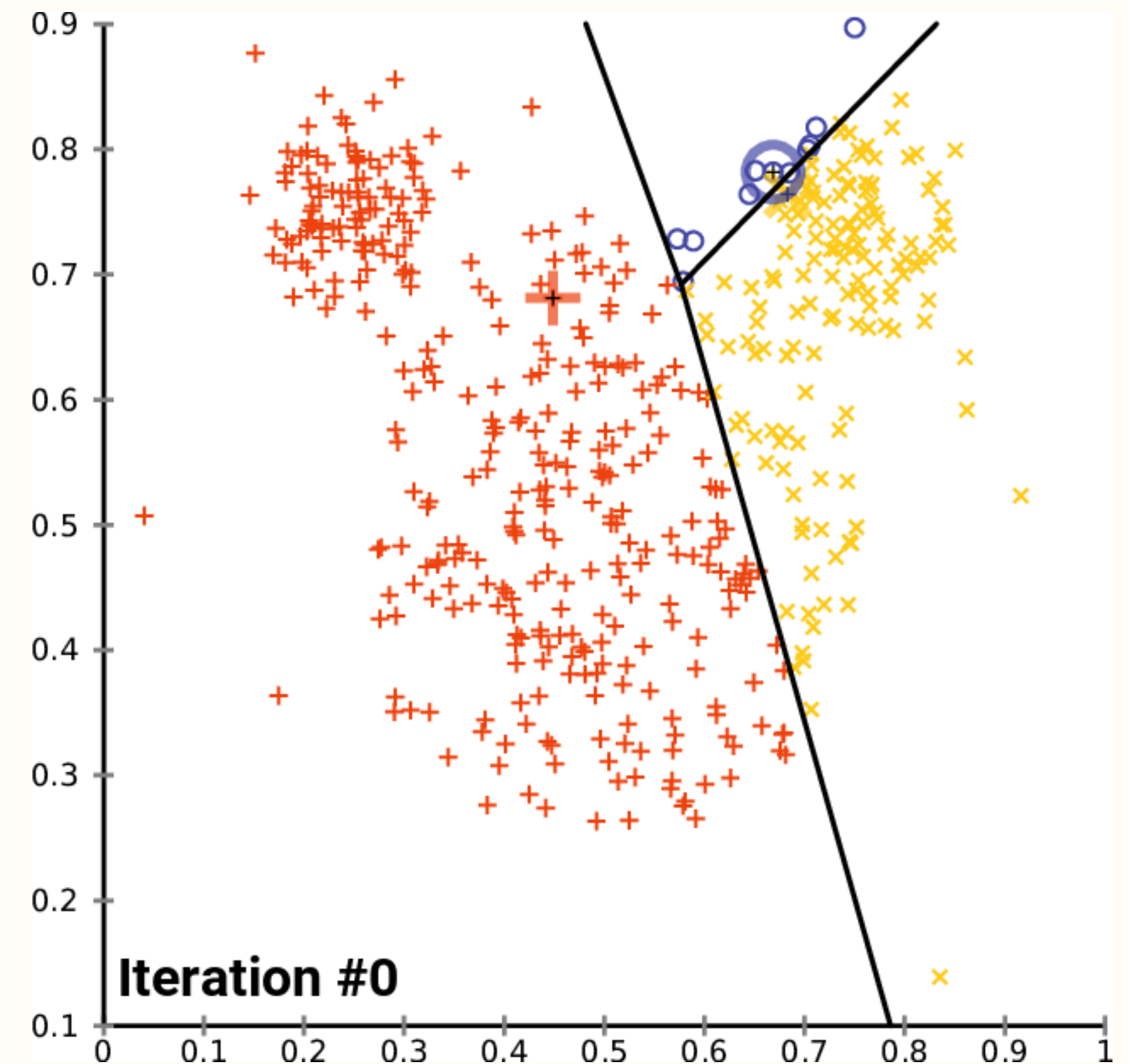
- The outcome y is not known
- There is no outcome to predict
- The goal is to **discover structure** or **patterns** in the data

K-means clustering

K-means clustering is an unsupervised ML method that partitions data into k groups (clusters) based on similarity.

Steps:

1. Initialize: start with k initial cluster centers (centroids).
2. Assignment: assign each observation to its nearest center.
3. Update: recompute each center as the mean of all observations assigned to that cluster.
4. Repeat steps 2–3 until the cluster assignments no longer change (or change very little).



Example: Pima Indians

Using the **Pima Indians diabetes dataset** from the **mlbench** package we will predict diabetes status, positive or negative for diabetes, using all other predictors available in the dataset.

- **pregnant**: Number of times pregnant
- **glucose**: Plasma glucose concentration (glucose tolerance test)
- **pressure**: Diastolic blood pressure (mm Hg)
- **triceps**: Triceps skin fold thickness (mm)
- **insulin**: 2-Hour serum insulin (mu U/ml)
- **mass**: Body mass index (BMI)
- **pedigree**: Diabetes pedigree function (a genetic risk score)
- **age**: Age (years)
- **diabetes**: indicator of diabetes

Example: Pima Indians

```
library(mlbench)
data("PimaIndiansDiabetes2")

PimaIndiansDiabetes2 <- PimaIndiansDiabetes2

head(PimaIndiansDiabetes2)
```

##	pregnant	glucose	pressure	triceps	insulin	mass	pedigree	age	diabetes
## 1	6	148	72	35	NA	33.6	0.627	50	pos
## 2	1	85	66	29	NA	26.6	0.351	31	neg
## 3	8	183	64	NA	NA	23.3	0.672	32	pos
## 4	1	89	66	23	94	28.1	0.167	21	neg
## 5	0	137	40	35	168	43.1	2.288	33	pos
## 6	5	116	74	NA	NA	25.6	0.201	30	neg

Missingness

K-means clustering is unable to handle categorical variables or any data missingness.

- Only use continuous or numerical variables for predictors
 - Other methods exist for categorical variables, like k-modes clustering
- Only keep complete cases for this example
 - Could also implement imputation as before, but will keep it simple for this example

Final sample

The predictors include glucose, pressure, triceps, insulin, mass, and age.

- The diabetes column will be kept in the complete cases data frame, but is not the outcome since this is unsupervised learning
- The diabetes labels will be used later to inspect the clusters (for educational purposes only)
- The final sample includes 392 individuals

```
# Select numeric variables for clustering (keep diabetes for later comparison)
pima_num <- PimaIndiansDiabetes2 %>%
  select(glucose, pressure, triceps, insulin, mass, age, diabetes)

# Remove rows with missing values in these variables
pima_complete <- pima_num %>%
  drop_na()

# Check how many observations remain
nrow(pima_num)
```

```
## [1] 768
```

```
nrow(pima_complete)
```

```
## [1] 392
```

Standardizing (scaling)

K-means clustering requires the predictor values to be on the same scale or to be standardized (scaled) to have mean 0 and standard deviation 1.

- The **scale** function will scale the predictors

```
# Data frame of numeric features (without the outcome)
pima_features <- pima_complete %>%
  select(glucose, pressure, triceps, insulin, mass, age)

# Scale the variables: each column to have mean 0, sd 1
pima_scaled <- scale(pima_features)

# Inspect summary of scaled variables
summary(pima_scaled)
```

```
##      glucose      pressure      triceps      insulin
## Min.      :-2.1590  Min.      :-3.73423  Min.      :-2.10579  Min.      :-1.1953
## 1st Qu.: -0.7656  1st Qu.: -0.69328  1st Qu.: -0.77454  1st Qu.: -0.6673
## Median :-0.1175  Median :-0.05308  Median :-0.01383  Median :-0.2571
## Mean   : 0.0000  Mean   : 0.00000  Mean   : 0.00000  Mean   : 0.0000
## 3rd Qu.: 0.6601  3rd Qu.: 0.58712  3rd Qu.: 0.74689  3rd Qu.: 0.2856
## Max.    : 2.4423  Max.    : 3.14792  Max.    : 3.21921  Max.    : 5.8056
##      mass      age
## Min.      :-2.11823  Min.      :-0.9671
## 1st Qu.: -0.66683  1st Qu.: -0.7710
## Median : 0.01619  Median :-0.3789
## Mean   : 0.00000  Mean   : 0.0000
## 3rd Qu.: 0.57114  3rd Qu.: 0.5034
## Max.    : 4.83999  Max.    : 4.9148
```


Number of clusters (k)

K-means clustering requires k to be specified in advance. There is no single “correct” way and three common ways include:

1. Subject matter knowledge: How many meaningful subgroups are expected clinically?
2. Elbow method: Plot the total within-cluster sum of squares (variability) versus k , and look for an “elbow”. Also known as a scree plot.
3. Silhouette scores and other criteria (more advanced).

Scree plot and elbow method

The scree plot is used for the “elbow” method.

- As k increases, the within-cluster variation (WSS) always decreases.
- Look for a point (value of k) where the decrease slows down – the “elbow.”
- If less obvious, try multiple values of k

```
set.seed(219) # for reproducibility

# Use the sapply function to apply this function to k = 1-8
wss <- sapply(1:8, function(k) {
  kmeans(pima_scaled, centers = k, nstart = 25)$tot.withinss
})

elbow_df <- data.frame(
  k = 1:8,
  wss = wss
)

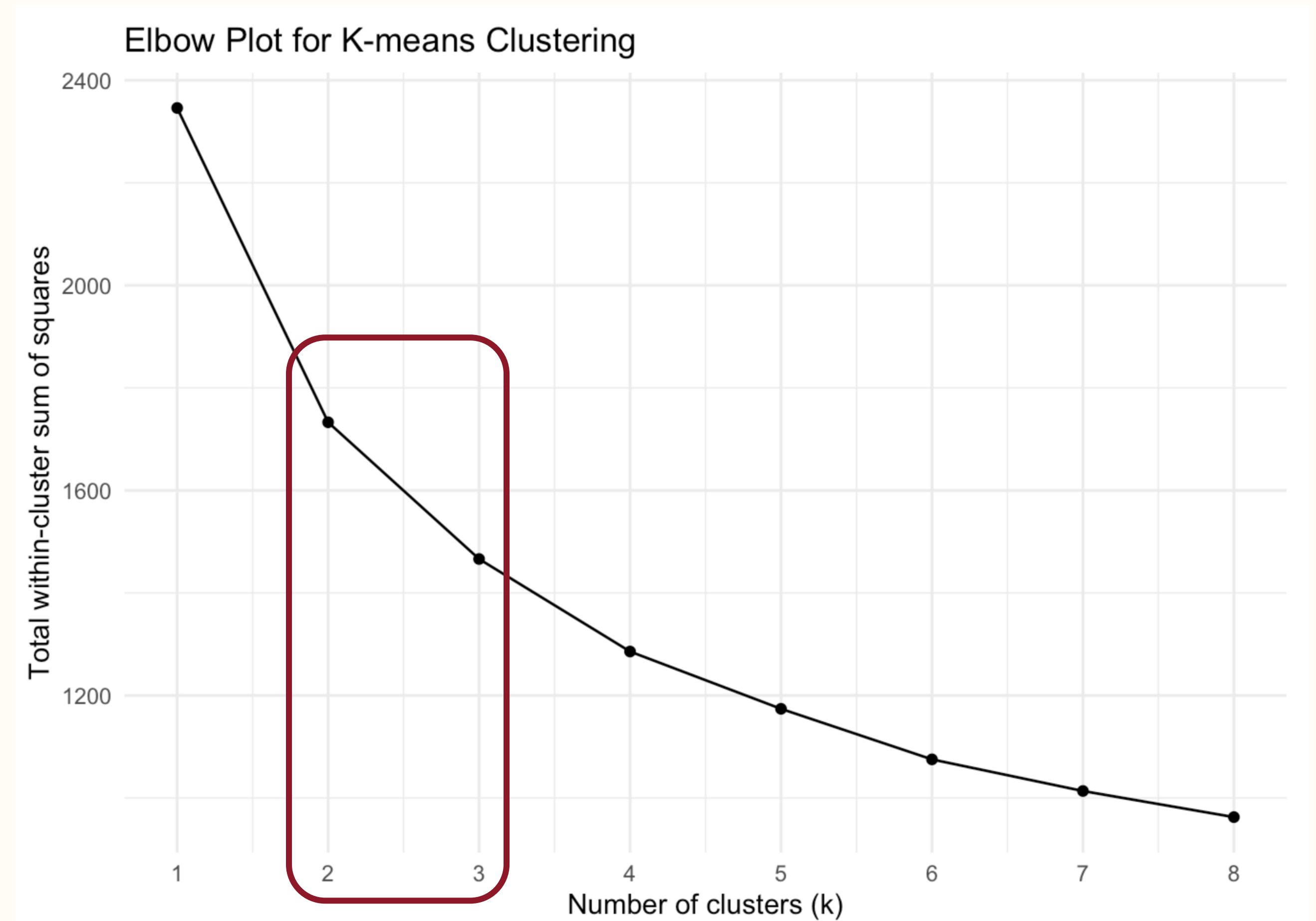
elbow_df
```

```
##   k      wss
## 1 1 2346.000
## 2 2 1732.891
## 3 3 1466.183
## 4 4 1285.709
## 5 5 1173.906
## 6 6 1075.267
## 7 7 1013.602
## 8 8  962.672
```


Scree plot and elbow method

The “elbow” is less obvious here. The slope starts to decrease at $k = 2$.

- Try $k = 2$ and $k = 3$



kmeans

The **kmeans** function on the scaled predictors is used to train the K-means model.

- **centers**: k , the number of clusters to create.
- **nstart**: the number of random initial configurations the algorithm will attempt. The function then reports the results from the run that has the lowest total within-cluster sum of squares (WSS), which is considered the optimal result.

```
set.seed(219)

k2 <- kmeans(pima_scaled, centers = 2, nstart = 25)

# Print summary of the k-means result
k2
```

kmeans

Key elements in the output:

- **cluster**: cluster assignments for each observation.
- **centers**: the cluster centers (means of each variable within clusters, on the scaled scale).
- **tot.withinss** and **betweenss**: measures of within- and between-cluster variation.

```
## K-means clustering with 2 clusters of sizes 228, 164
##
## Cluster means:
##      glucose  pressure  triceps  insulin  mass  age
## 1 -0.4863497 -0.4174023 -0.4438142 -0.3935206 -0.4105078 -0.4403575
## 2  0.6761446  0.5802909  0.6170100  0.5470896  0.5707059  0.6122044
##
## Clustering vector:
##  4  5  7  9 14 15 17 19 20 21 25 26 28 29 32 33 36 40 41 44
##  1  1  1  2  2  2  2  1  1  2  2  1  1  2  2  1  1  2  1  2
```

```
## Within cluster sum of squares by cluster:
## [1] 741.2004 991.6903
## (between_SS / total_SS = 26.1 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```


Interpreting summaries with clusters

Cluster centers from **kmeans** are on the *standardized* scale. A more interpretable summary is to compute the mean of each original variable within each cluster.

```
pima_k2 <- pima_complete %>%
  mutate(cluster = factor(k2$cluster))

# How many observations per cluster?
pima_k2 %>%
  count(cluster)
```

```
##   cluster    n
## 1         1 228
## 2         2 164
```

```
cluster_means_k2 <- pima_k2 %>%
  group_by(cluster) %>%
  summarize(
    glucose = mean(glucose, na.rm = TRUE),
    pressure = mean(pressure, na.rm = TRUE),
    triceps = mean(triceps, na.rm = TRUE),
    insulin = mean(insulin, na.rm = TRUE),
    mass = mean(mass, na.rm = TRUE),
    age = mean(age, na.rm = TRUE),
    .groups = "drop"
  )
```

cluster_means_k2

```
## # A tibble: 2 × 7
##   cluster glucose pressure triceps insulin mass age
##   <fct>    <dbl>    <dbl>   <dbl>   <dbl> <dbl> <dbl>
## 1 1         108.     65.4    24.5    109.  30.2  26.4
## 2 2         143.     77.9    35.6    221.  37.1  37.1
```


Interpreting summaries with clusters

Looking at the means for each cluster, cluster 2 individuals have

- Higher mean glucose
- Higher mean diastolic blood pressure
- Higher mean triceps skin fold thickness
- Higher mean insulin
- Higher mean BMI
- Higher mean age

All are related to a higher probability of having diabetes, which makes sense clinically.

```
cluster_means_k2 <- pima_k2 %>%  
  group_by(cluster) %>%  
  summarize(  
    glucose = mean(glucose, na.rm = TRUE),  
    pressure = mean(pressure, na.rm = TRUE),  
    triceps = mean(triceps, na.rm = TRUE),  
    insulin = mean(insulin, na.rm = TRUE),  
    mass = mean(mass, na.rm = TRUE),  
    age = mean(age, na.rm = TRUE),  
    .groups = "drop"  
  )
```

cluster_means_k2

```
## # A tibble: 2 × 7  
##   cluster glucose pressure triceps insulin mass age  
##   <fct>      <dbl>    <dbl>   <dbl>   <dbl> <dbl> <dbl>  
## 1 1         108.     65.4    24.5    109.  30.2  26.4  
## 2 2         143.     77.9    35.6    221.  37.1  37.1
```


Visualizing clusters with PCA

Due to multiple numeric variables, clusters cannot be directly visualized in the original feature space.

A common strategy for visualization is to:

1. Perform principal component analysis (PCA) on the scaled features.
2. Plot the first two principal components and color points by cluster.

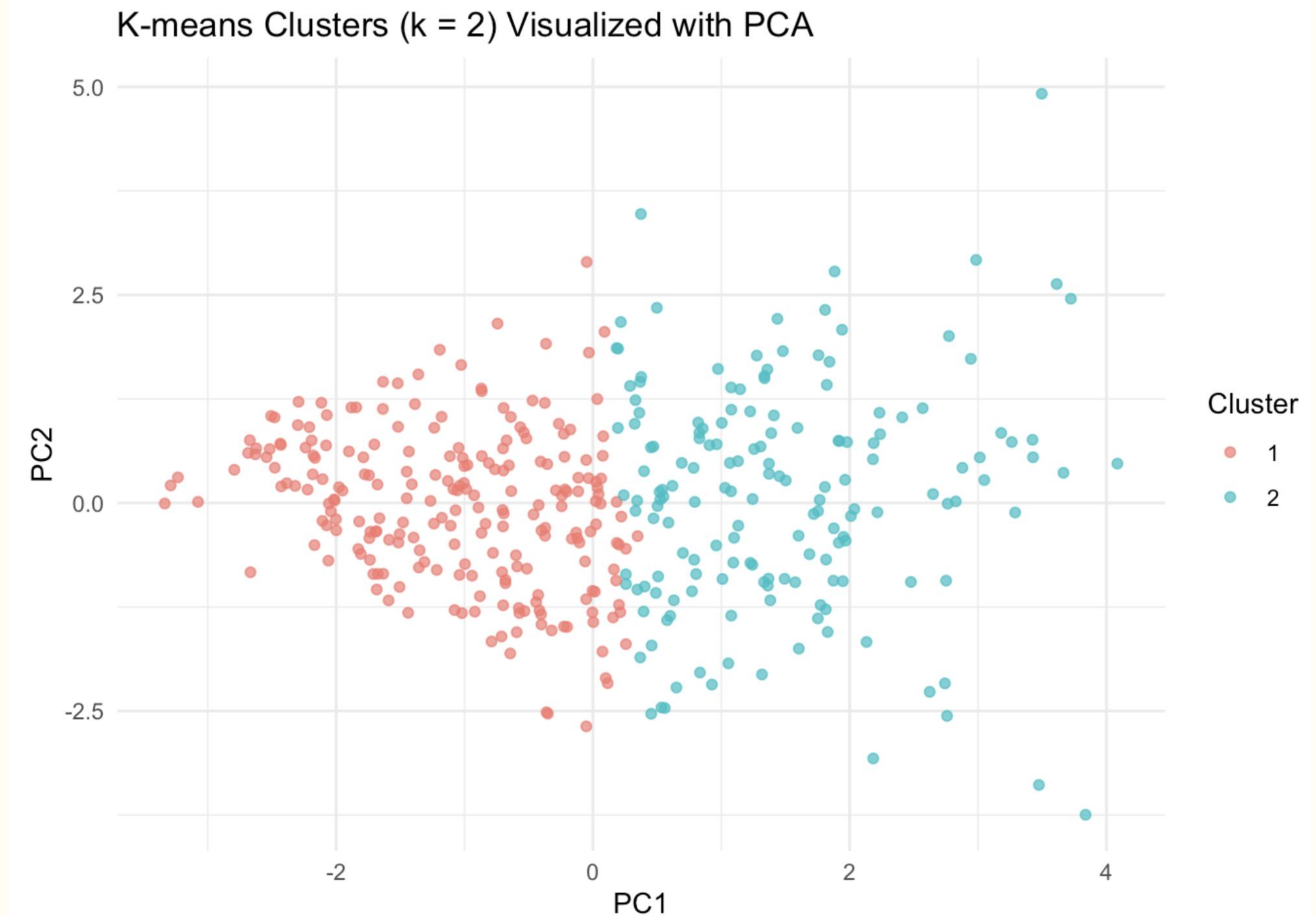
This is not part of *k*-means itself, but it provides a helpful visualization.

Visualizing clusters with PCA

```
# Don't need to scale here since the data is already scaled
pca <- prcomp(pima_scaled, center = FALSE, scale. = FALSE)

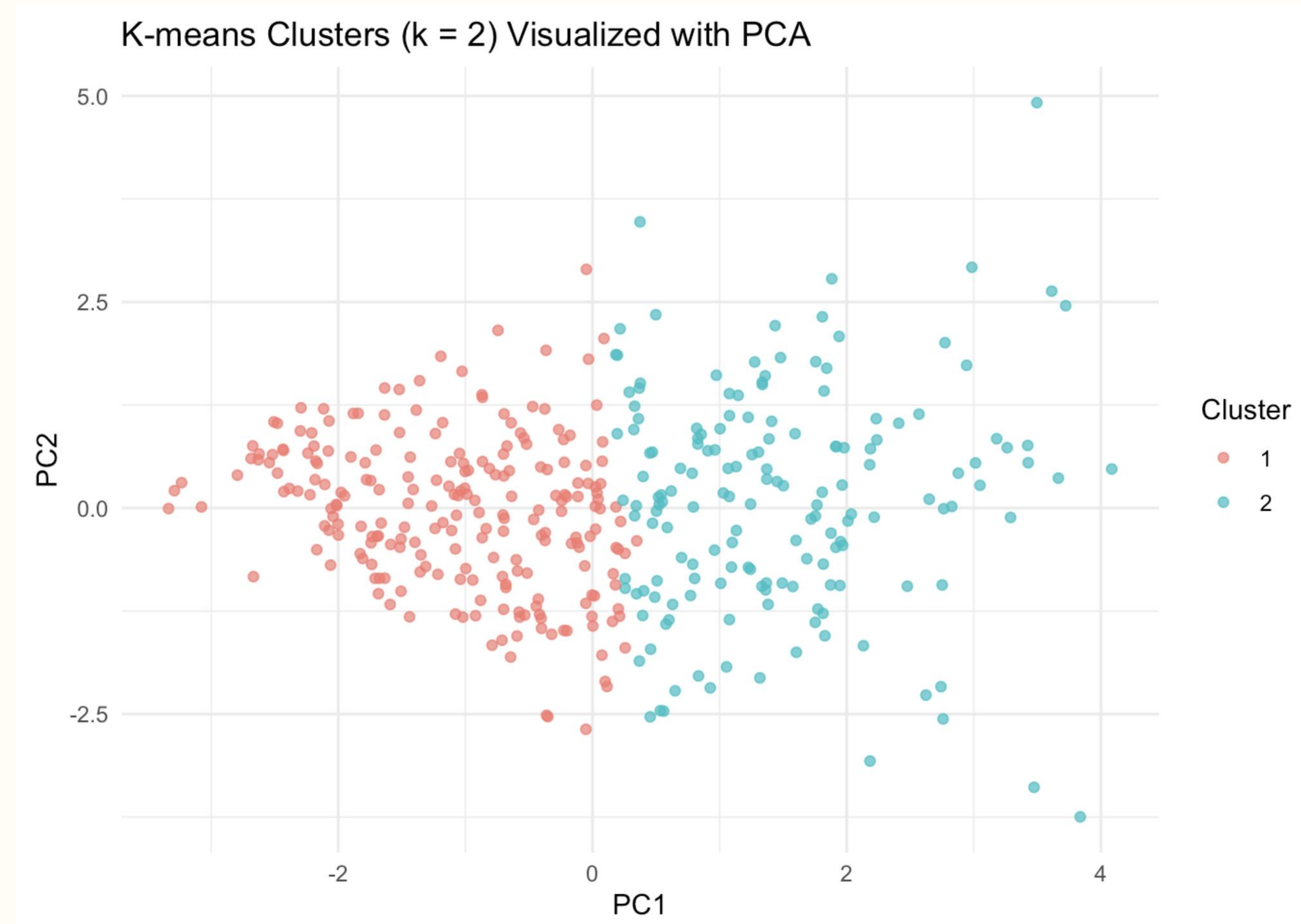
pca_df_k2 <- data.frame(
  PC1 = pca$x[, 1],
  PC2 = pca$x[, 2],
  cluster = pima_k2$cluster,
  diabetes = pima_k2$diabetes
)

ggplot(pca_df_k2, aes(x = PC1, y = PC2, color = cluster)) +
  geom_point(alpha = 0.7) +
  labs(title = "K-means Clusters (k = 2) Visualized with PCA",
       color = "Cluster") +
  theme_minimal()
```



Visualizing clusters with PCA

- Points that are closer together are more similar in terms of the selected clinical variables.
- If clusters overlap heavily in this plot, the separation may not be very strong, i.e., the individuals may not be very different between the clusters



Relating clusters to diabetes status

Although k-means clustering is unsupervised, we can compare clusters to the diabetes outcome as a post-hoc analysis since we have this information.

- This does **not** mean *k*-means is “predicting diabetes,” but it can tell us whether the clusters correspond to different diabetes prevalence.

```
cluster_diabetes_tab_k2 <- pima_k2 %>%  
  count(cluster, diabetes) %>%  
  group_by(cluster) %>%  
  mutate(prop = n / sum(n))
```

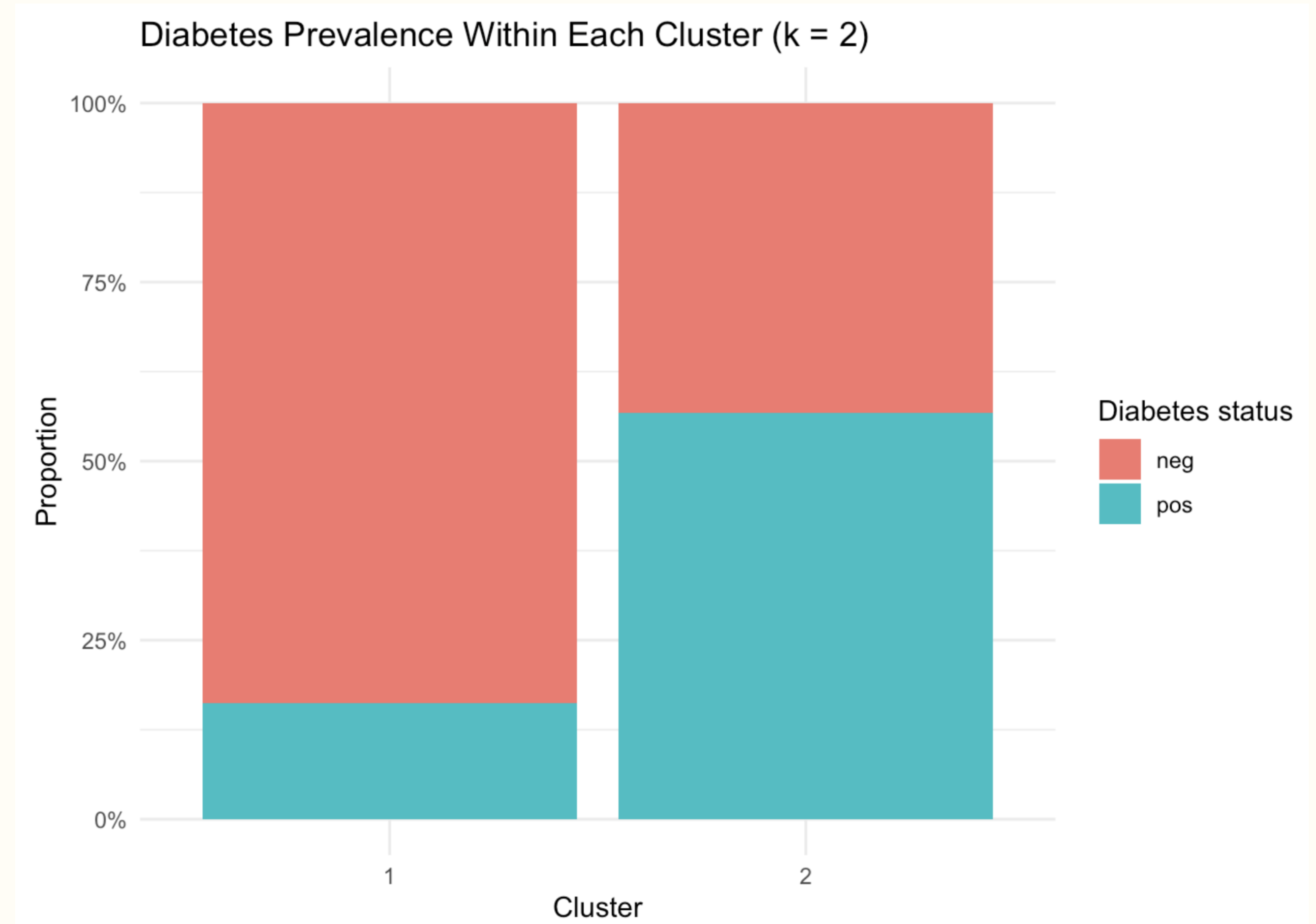
```
cluster_diabetes_tab_k2
```

```
## # A tibble: 4 × 4  
## # Groups:   cluster [2]  
##   cluster diabetes      n  prop  
##   <fct>    <fct>    <int> <dbl>  
## 1 1      neg      191 0.838  
## 2 1      pos       37 0.162  
## 3 2      neg       71 0.433  
## 4 2      pos       93 0.567
```

Relating clusters to diabetes status

Although k-means clustering is unsupervised, we can compare clusters to the diabetes outcome as a post-hoc analysis since we have this information.

- Cluster 2 does have a higher proportion of individuals who have been diagnosed with diabetes.



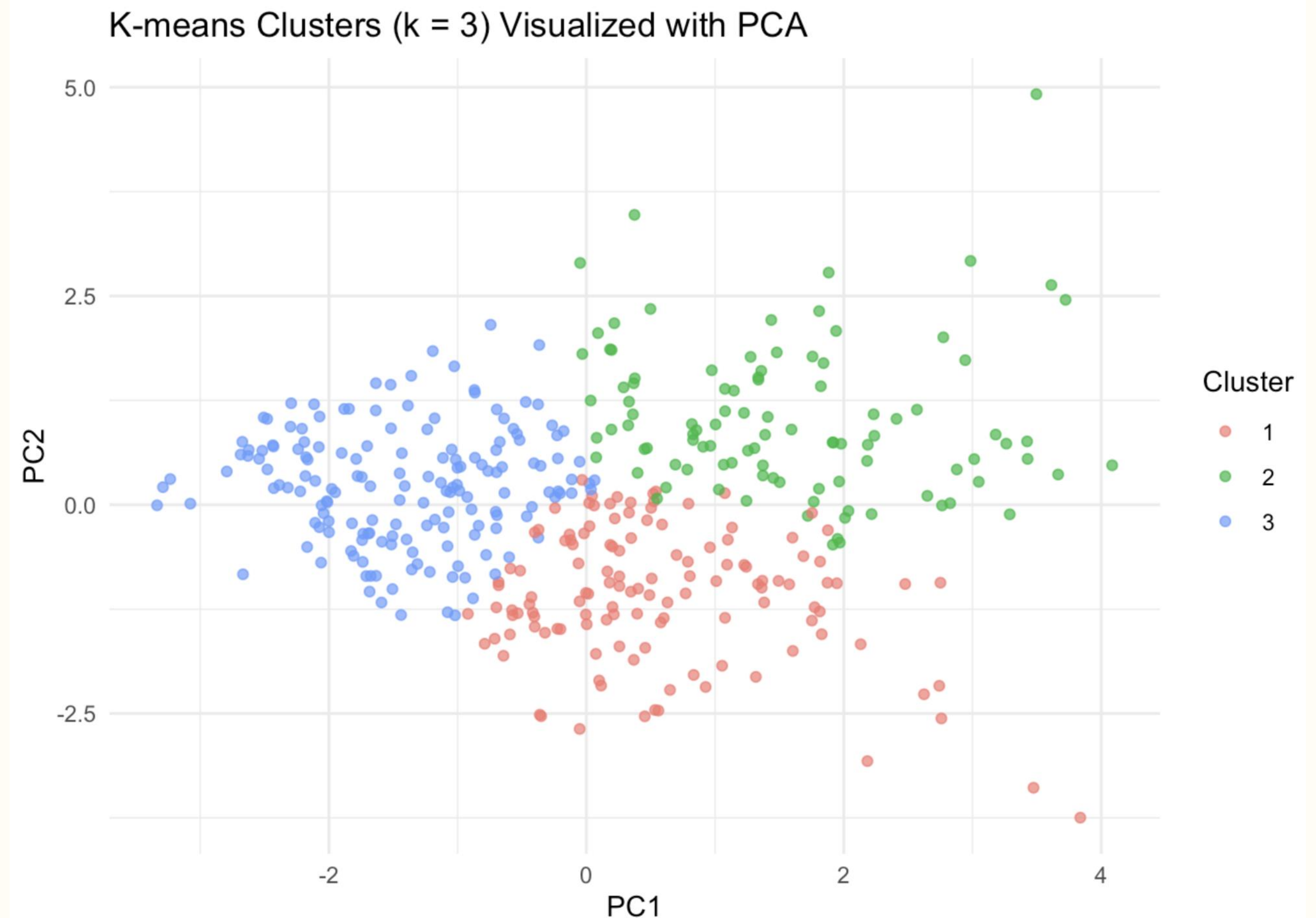
Trying $k = 3$ clusters

Repeat the process with $k = 3$ clusters to see whether it yields more interpretable or clinically meaningful groups.

```
cluster_means_k3 <- pima_k3 %>%  
  group_by(cluster) %>%  
  summarize(  
    glucose = mean(glucose, na.rm = TRUE),  
    pressure = mean(pressure, na.rm = TRUE),  
    triceps = mean(triceps, na.rm = TRUE),  
    insulin = mean(insulin, na.rm = TRUE),  
    mass = mean(mass, na.rm = TRUE),  
    age = mean(age, na.rm = TRUE),  
    .groups = "drop"  
  )
```

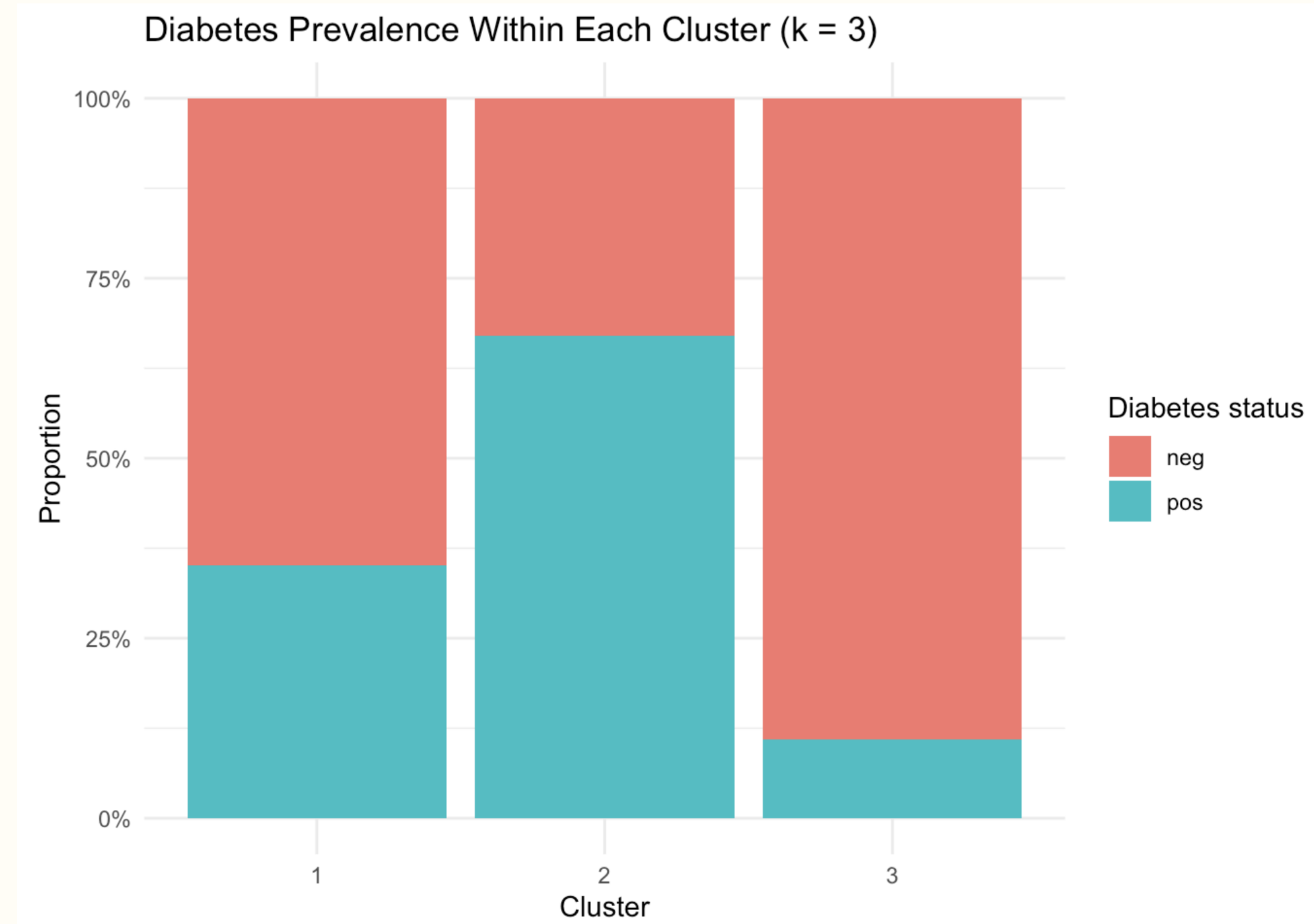
cluster_means_k3

```
## # A tibble: 3 × 7  
##   cluster glucose pressure triceps insulin mass age  
##   <fct>    <dbl>    <dbl>   <dbl>   <dbl> <dbl> <dbl>  
## 1 1      113.     73.9    37.9    130.  38.6  30.6  
## 2 2      162.     75.5    31.3    274.  34.4  39.1  
## 3 3      106.     65.2    21.0    104.  27.9  26.0
```



Trying $k = 3$ clusters

- Cluster 3 has the least number of individuals with diabetes and cluster 2 has the most individuals with diabetes.
- Cluster 3 also has the lowest values for all predictors compared to clusters 1 and 2



K-means clustering summary

K-means Strengths

- Simple and fast to run.
- Works well for moderately sized numeric datasets.
- Produces clusters that are often easy to summarize.

K-means Limitations

- k must be pre-specified.
- Sensitive to scaling and outliers.
- -Assumes clusters are roughly “spherical” in Euclidean space and similarly sized.
- Can converge to different solutions depending on starting values (mitigated by **nstart**).

Key takeaways

- Unsupervised learning methods like K-means clustering do not use an outcome variable; they aim to discover patterns directly from predictors.
- K-means partitions observations into k groups by minimizing within-cluster variation.
- Proper preprocessing (handling missing data and scaling variables) is crucial for meaningful clusters.
- The choice of k is an important modeling decision; simple tools such as the elbow method can guide this choice.
- In a public health or clinical setting, clusters can sometimes correspond to risk profiles or subgroups with different disease prevalence or health outcomes.
- Clusters should be interpreted carefully and (ideally) validated using domain knowledge and external evidence.